CAHIER DES CHARGES

MediScan AI

Application Web de Santé Intelligente

Interface Patient & Interface Médecin · IA de Reconnaissance d'Image · Analyse d'Anamnèse

Version 2.0 — passage du développement mobile au développement web

Date de révision : 1er novembre 2026 (version 1.0 du 17 août 2026)

Stack technique : Angular (frontend web) · Spring Boot (backend) · MySQL · Postman · Firebase Cloud Messaging (notifications)

1. Présentation du projet

MediScan AI est une **application web** de santé qui vise à simplifier et fiabiliser la collecte de l'anamnèse des patients et l'identification des médicaments, grâce à l'intelligence artificielle (reconnaissance d'image / OCR et traitement du langage naturel). L'application, accessible depuis un navigateur sur ordinateur, tablette ou smartphone, propose deux interfaces distinctes : une interface Patient et une interface Médecin, connectées à un moteur IA central.

Thème du projet. Création d'une application web avec une interface patient et une interface médecin, qui a pour objectif de traduire les données personnelles des patients (anamnèse) en données exploitables pour les médecins via l'intelligence artificielle (IA). Le second volet porte sur la reconnaissance d'image des médicaments : le patient importe ou capture, via la webcam ou l'appareil photo de son navigateur, la photo du packaging, et l'IA en extrait automatiquement les informations clés.

Ce document constitue le cahier des charges du projet : il définit le contexte, les objectifs, le périmètre du MVP (Minimum Viable Product), les exigences fonctionnelles et non fonctionnelles, l'architecture technique retenue, ainsi que les diagrammes de conception (cas d'utilisation, classes, séquence) et le planning de développement (Gantt).

Note de révision : la version 1.0 prévoyait un frontend mobile Android. La version 2.0 remplace ce frontend par une application web Angular. Le backend Spring Boot, la base MySQL, les tests Postman et le service IA restent inchangés.

1.1 Contexte et problématique

- **Données fragmentées** : l'anamnèse des patients est dispersée dans différents systèmes et formats non standardisés.
- **Médicaments illisibles** : les packagings contiennent des informations clés difficiles à saisir manuellement.
- **Temps médical précieux** : les médecins consacrent trop de temps à la collecte d'informations plutôt qu'au diagnostic.
- **Manque d'exploitation IA** : les données brutes des patients ne sont pas transformées en insights cliniques exploitables.

1.2 Objectifs du projet

- Simplifier la saisie de l'anamnèse patient via un formulaire web intelligent guidé.
- Automatiser l'extraction des informations médicamenteuses par reconnaissance d'image (OCR + IA).
- Fournir aux médecins un dashboard clinique web avec données structurées et alertes intelligentes.
- Garantir la sécurité, la confidentialité et la conformité réglementaire (RGPD / HDS).
- Rendre l'application accessible sans installation, depuis tout navigateur récent (design responsive).
- Livrer un MVP fonctionnel démontrant l'intégration de bout en bout : navigateur (Angular) ↔ API ↔ base de données ↔ IA.

2. Périmètre du MVP

Le MVP se concentre sur le socle fonctionnel indispensable pour démontrer la valeur du produit, sans les fonctionnalités avancées (certification CE, API hôpitaux, déploiement national), prévues dans les phases ultérieures de la feuille de route.

2.1 Fonctionnalités incluses dans le MVP

- Authentification (inscription / connexion) Patient et Médecin, avec routes Angular protégées par rôle.

- Interface Patient : saisie d'anamnèse guidée, profil, historique.
- Scan de médicament : import d'une photo (fichier) ou capture via la webcam du navigateur, puis extraction IA (nom, dosage, posologie, contre-indications).
- Vérification des données extraites contre une base médicamenteuse (simulée ou API publique).
- Rappels médicamenteux : planification côté backend et notifications Web Push via Firebase Cloud Messaging.
- Interface Médecin : dashboard listant les patients, fiches synthétiques, alertes d'interaction basiques.
- Partage sécurisé des données du patient vers un médecin (consentement explicite).
- Persistance des données dans MySQL via une API REST Spring Boot testée avec Postman.

2.2 Fonctionnalités hors périmètre MVP (phases ultérieures)

- Aide au diagnostic avancée / IA explicable conforme EU AI Act.
- Certification CE dispositif médical (IEC 62304, MDR 2017/745) et hébergement HDS certifié.
- Intégration API avec systèmes hospitaliers et interopérabilité nationale.
- Audit et traçabilité complets, chiffrement end-to-end avancé.
- Version Progressive Web App (PWA) installable et mode hors ligne.

3. Acteurs du système

Acteur	Description	Interactions principales
Patient	Utilisateur final de l'application web souhaitant gérer sa santé depuis son navigateur.	Saisie anamnèse, scan médicament, historique, rappels, partage médecin
Médecin	Professionnel de santé consultant les données de ses patients via le dashboard web.	Dashboard, fiches patients, alertes, aide à la décision
Service IA	Moteur d'intelligence artificielle (OCR, NLP) intégré au backend.	Analyse d'image, extraction de données, détection d'interactions
Base médicamenteuse	Référentiel externe des médicaments (national / public).	Vérification et enrichissement des données extraites
Firebase (FCM)	Service externe utilisé uniquement pour l'envoi des notifications.	Rappels de prise, nouvelles alertes, notifications de partage

4. Exigences fonctionnelles

Réf.	Exigence	Priorité
EF-01	Le système doit permettre à un utilisateur (patient/médecin) de créer un compte et de s'authentifier depuis le navigateur.	Haute
EF-02	Le patient doit pouvoir saisir et modifier son anamnèse via un formulaire web guidé (étapes successives).	Haute
EF-03	Le patient doit pouvoir importer ou capturer (webcam) la photo d'un médicament et obtenir automatiquement ses données (nom, DCI, dosage, posologie, contre-indications, lot, expiration).	Haute
EF-04	Le système doit vérifier les données extraites par rapport à une base médicamenteuse de référence.	Haute
EF-05	Le patient doit pouvoir consulter l'historique de ses scans et de son anamnèse.	Moyenne

Réf.	Exigence	Priorité
EF-06	Le patient doit pouvoir configurer des rappels de prise médicamenteuse et les recevoir sous forme de notifications web.	Moyenne
EF-07	Le patient doit pouvoir partager ses données avec un médecin via consentement explicite, et le révoquer.	Haute
EF-08	Le médecin doit disposer d'un dashboard web listant ses patients et leurs indicateurs clés.	Haute
EF-09	Le système doit générer des alertes automatiques en cas d'interaction médicamenteuse ou de contre-indication.	Haute
EF-10	Le médecin doit pouvoir consulter la fiche synthétique et l'historique complet d'un patient.	Moyenne

5. Exigences non fonctionnelles

- **Sécurité** : mots de passe hashés (BCrypt), authentification par token (JWT) transmis dans l'en-tête Authorization par un intercepteur HTTP Angular, communications chiffrées (HTTPS/TLS), configuration CORS restreinte à l'origine du frontend.
- **Sécurité web** : protection contre les failles XSS (échappement natif d'Angular), validation des données côté client (formulaires réactifs) et côté serveur (Bean Validation).
- **Confidentialité** : consentement explicite du patient avant tout partage de données médicales.
- **Performance** : temps de réponse de l'API < 2 secondes pour les opérations courantes ; traitement OCR/IA < 5 secondes ; chargement initial de l'application web < 3 secondes.
- **Compatibilité** : fonctionnement sur les dernières versions de Chrome, Firefox, Edge et Safari ; interface responsive (ordinateur, tablette, smartphone).
- **Disponibilité** : architecture backend découplée, prête pour un déploiement conteneurisé (Docker) ultérieur ; frontend servi en fichiers statiques (Nginx).
- **Ergonomie** : interface intuitive, accessible (bonnes pratiques WCAG), adaptée à un public non technique (patients de tous âges).
- **Portabilité** : API REST documentée, testable indépendamment du frontend via Postman.
- **Maintenabilité** : architecture en couches (Controller / Service / Repository) côté Spring Boot, architecture modulaire (composants, services, guards) côté Angular, code versionné (Git).

6. Architecture technique

L'application repose sur une architecture client-serveur classique en trois niveaux, adaptée à un développement académique/MVP réalisable par un développeur ou une petite équipe :

Couche	Technologie	Rôle
Frontend web	Angular (TypeScript), Angular Material, VS Code	Interfaces Patient et Médecin, appels HTTP vers l'API REST via HttpClient
Backend / API	Spring Boot (Java), IntelliJ IDEA	Logique métier, sécurité (Spring Security, JWT), exposition des endpoints REST
Base de données	MySQL	Stockage persistant (utilisateurs, anamnèse, médicaments, scans, alertes, rappels, partages)
Tests API	Postman	Validation manuelle et automatisée des endpoints REST

Couche	Technologie	Rôle
Service IA	Module Spring Boot dédié (ou microservice Python appelé en interne)	OCR / extraction d'informations depuis l'image du médicament
Notifications	Firebase Cloud Messaging (Web Push)	Envoi des rappels et alertes au navigateur, uniquement pour les notifications

Flux général : l'application Angular, exécutée dans le navigateur, envoie des requêtes HTTP/JSON (ou multipart pour les images) à l'API Spring Boot ; celle-ci orchestre la logique métier, interroge le service IA pour l'analyse d'image, persiste les données dans MySQL via Spring Data JPA, et retourne les résultats structurés au client web. Les notifications sont envoyées par le backend via le SDK Firebase Admin et reçues par le service worker du navigateur. Chaque endpoint est validé et documenté via des collections Postman avant intégration côté Angular.

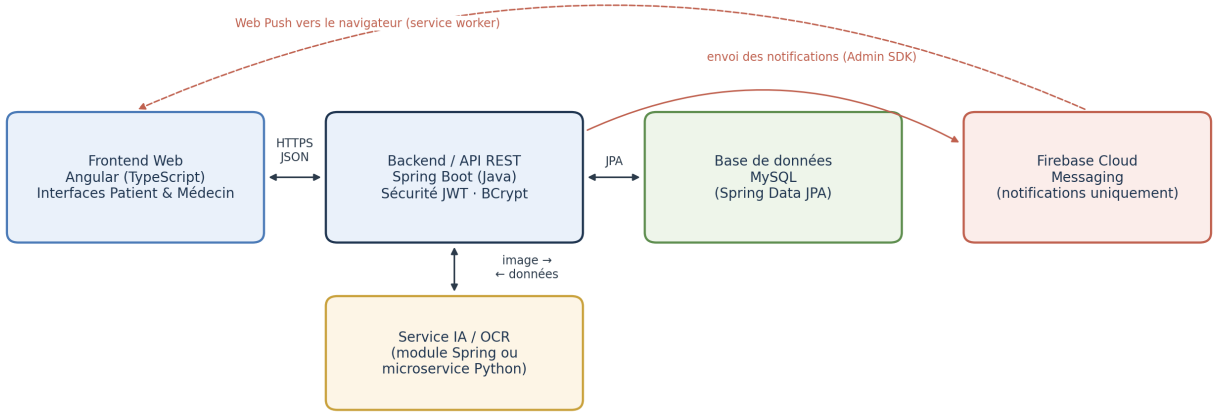


Figure 1 — Architecture technique de MediScan AI (version web)

7. Diagramme de cas d'utilisation

Le diagramme ci-dessous présente les principales interactions entre les acteurs (Patient, Médecin) et le système MediScan AI, ainsi que les relations d'inclusion (<<include>>) et d'extension (<<extend>>) entre cas d'utilisation. Il reste valable pour la version web : seul le canal d'accès change.

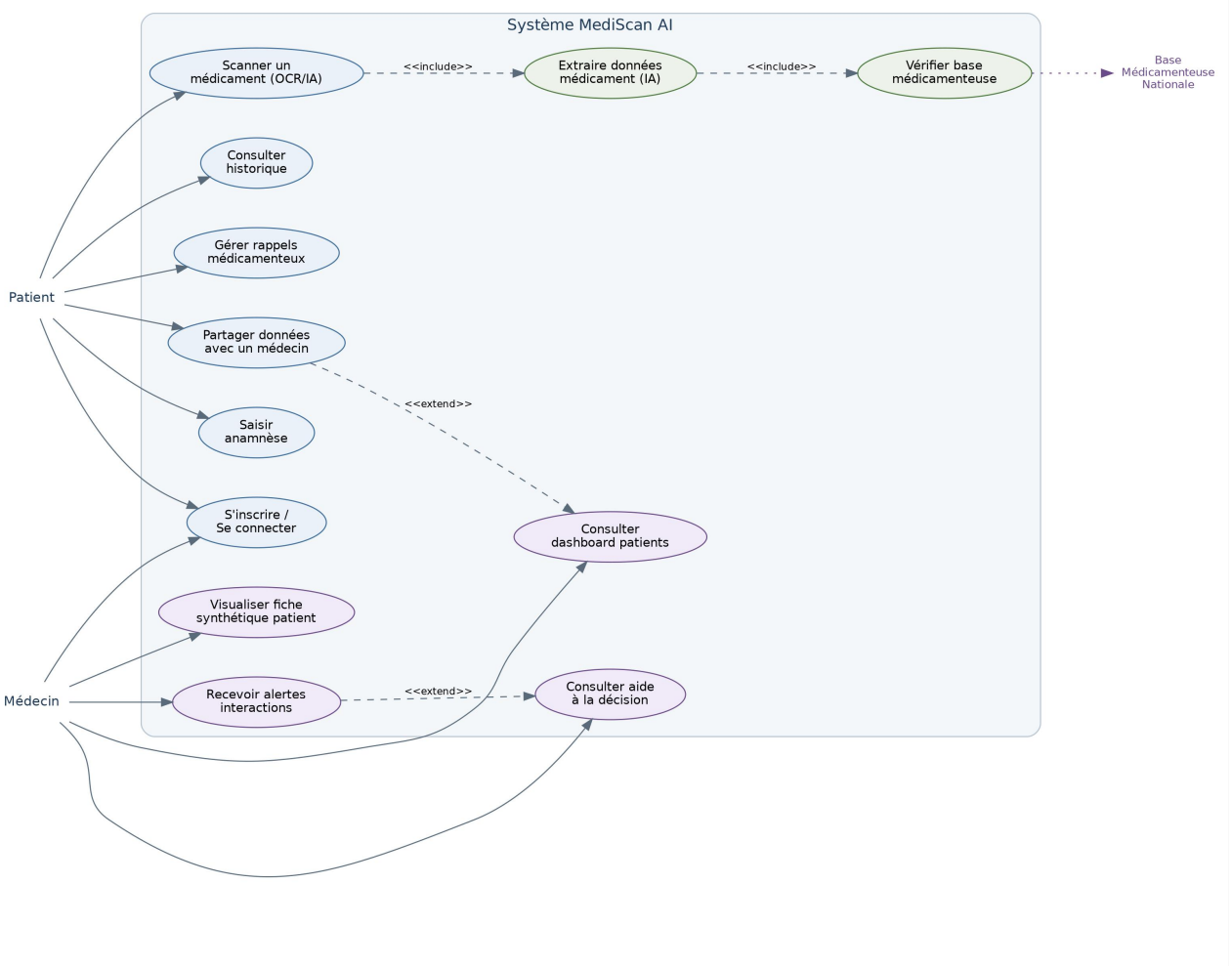


Figure 2 — Diagramme de cas d'utilisation MediScan AI

8. Diagramme de classes

Ce diagramme modélise les principales entités métier du système et leurs relations : utilisateurs (patients, médecins), anamnèse, médicaments, scans, alertes, rappels, partages de données, ainsi que les services techniques (Service IA, Base médicamenteuse externe). Le modèle métier est indépendant du frontend et ne change pas avec le passage au web.

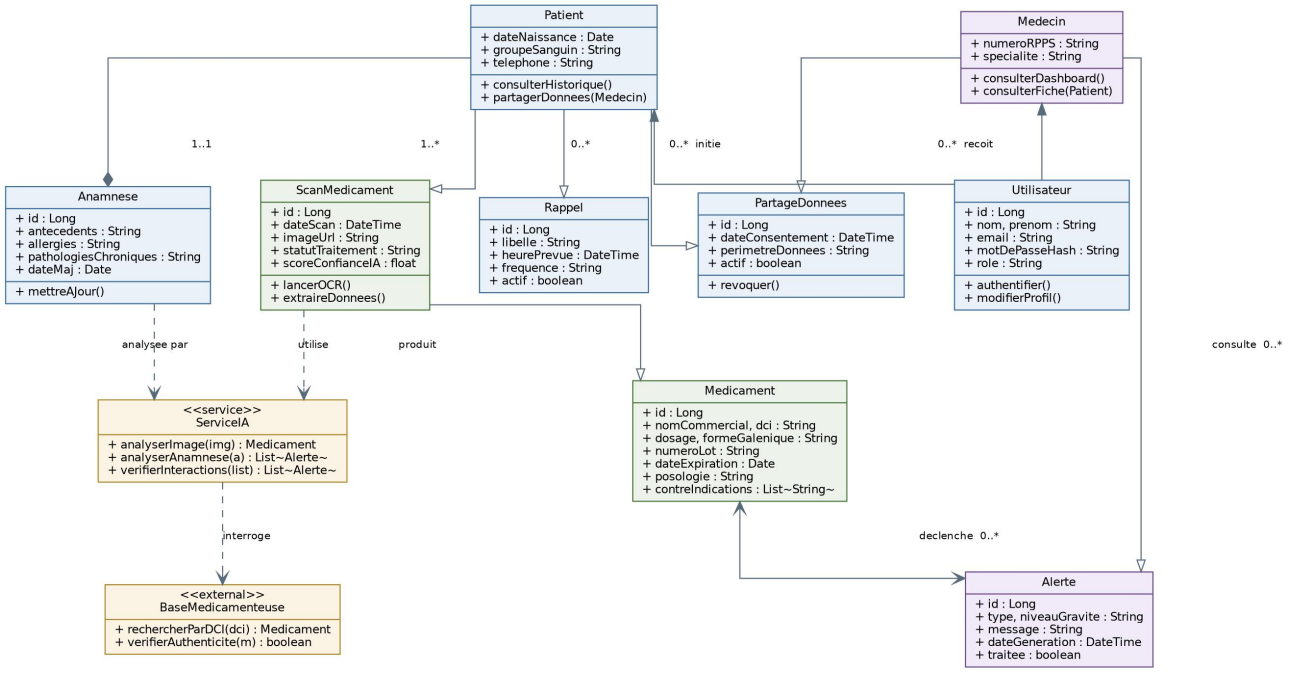


Figure 3 — Diagramme de classes MediScan AI

9. Diagramme de séquence

Le scénario ci-dessous illustre le flux principal du MVP dans sa version web : le patient importe ou capture la photo d'un médicament dans son navigateur, l'image est traitée par le service IA, vérifiée contre la base médicamenteuse nationale, persistée en base MySQL, et le résultat structuré (avec alerte éventuelle et notification Web Push) est renvoyé à l'application Angular.

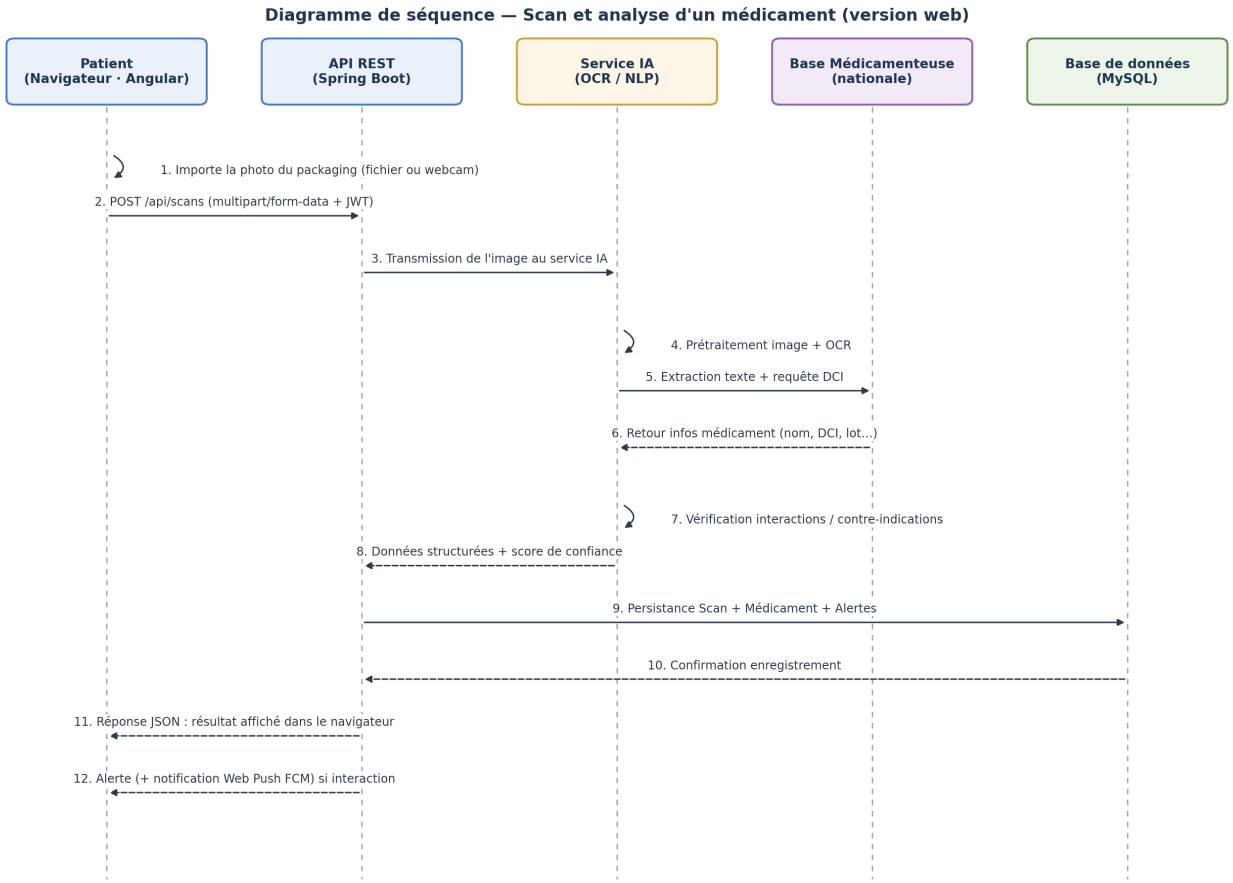


Figure 4 — Diagramme de séquence : scan et analyse d'un médicament (version web)

10. Planning prévisionnel (diagramme de Gantt)

Remarque : le planning conserve les jalons de la version 1.0 (17 août 2026 → 15 mars 2027). Les tâches de développement frontend portent désormais sur Angular au lieu d'Android. La ligne rouge indique la date de révision (1er novembre 2026). Le planning est à ajuster selon la disponibilité réelle de l'équipe et l'avancement constaté.

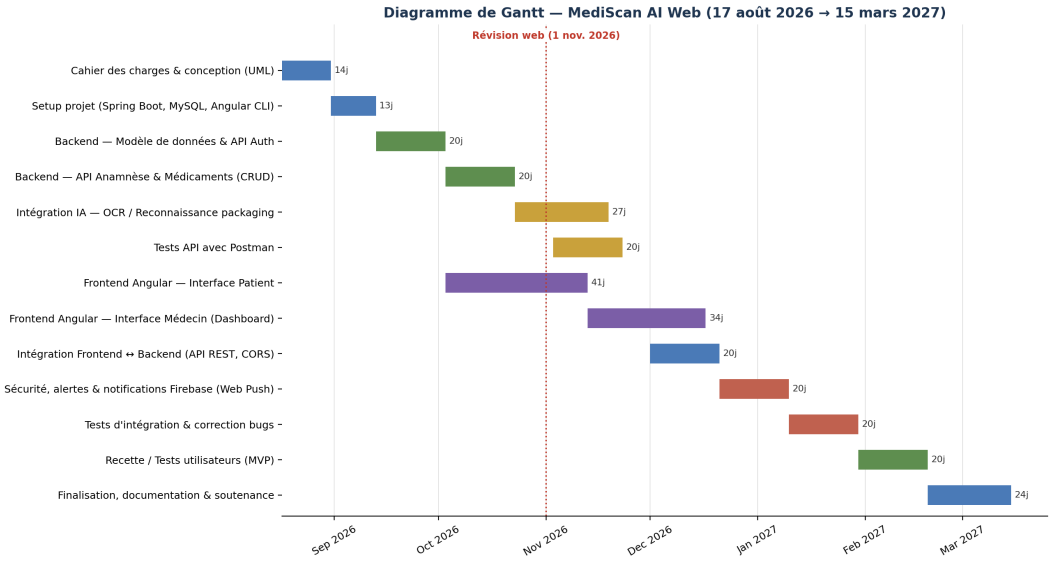


Figure 5 — Diagramme de Gantt prévisionnel du projet MediScan AI (version web)

10.1 Jalons clés

Jalon	Échéance indicative	Livrable
Fin conception	31 août 2026	Cahier des charges + diagrammes UML validés
Backend fonctionnel	26 octobre 2026	API REST Auth + Anamnèse + Médicaments opérationnelle, testée sur Postman
IA intégrée	23 novembre 2026	Reconnaissance d'image opérationnelle et connectée à l'API
Frontend Angular complet	21 décembre 2026	Interfaces web Patient et Médecin fonctionnelles
Intégration complète	21 décembre 2026	Application web connectée de bout en bout à l'API
MVP livré	15 mars 2027	Application testée, recette effectuée, documentation livrée

11. Livrables attendus

- Cahier des charges et diagrammes UML (ce document).
- Code source backend Spring Boot (repository Git) + scripts SQL de création de la base MySQL.
- Collection Postman documentée pour tous les endpoints de l'API.
- Code source frontend Angular (interfaces Patient et Médecin).
- Configuration Firebase Cloud Messaging (service worker, clés VAPID) pour les notifications.
- Documentation technique (architecture, guide d'installation, guide d'utilisation).
- Application web MVP fonctionnelle, démontrant le flux complet : inscription → anamnèse → scan médicament → dashboard médecin.